

TASK 3 – DBMS

Seller and Inventory Management System

1. Aim

To design a MySQL-based Seller and Inventory Management System that organizes supplier details, item records, warehouse stock, and availability information in a structured database.

2. Objectives

- Maintain seller and supplier contact information.
- Store product descriptions, categories, and selling prices.
- Associate each product with its corresponding seller.
- Track stock quantities and reorder levels.
- Classify products according to their current stock condition.
- Perform insertion, updating, deletion, and retrieval operations.
- Generate useful stock summaries using SQL queries.

3. Database Design

The database contains three connected tables:

Seller Table

- Seller ID
- Seller Name
- Phone Number
- Email Address

Product Table

- Product ID
- Product Name
- Category
- Price
- Seller ID

Inventory Table

- Inventory ID
- Product ID
- Stock Quantity
- Reorder Level
- Stock Status

4. Relationship Between Tables

Seller → Product → Inventory

One seller may provide several products. Each product is connected to one inventory record that stores its current stock information and reorder threshold.

5. Create Database

```
CREATE DATABASE store_inventory_db;  
USE store_inventory_db;
```

6. Create Seller Table

```
CREATE TABLE Seller (  
    seller_id INT PRIMARY KEY,  
    seller_name VARCHAR(60),  
    phone VARCHAR(15),  
    email VARCHAR(70)  
);
```

7. Create Product Table

```
CREATE TABLE Product (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(60),  
    category VARCHAR(40),  
    price DECIMAL(10,2),  
    seller_id INT,  
    FOREIGN KEY (seller_id) REFERENCES Seller(seller_id)  
);
```

8. Create Inventory Table

```
CREATE TABLE Inventory (  
    inventory_id INT PRIMARY KEY,  
    product_id INT,  
    stock_quantity INT,  
    reorder_level INT,  
    status VARCHAR(20),  
    FOREIGN KEY (product_id) REFERENCES Product(product_id)  
);
```

9. Insert Seller Records

```
INSERT INTO Seller VALUES
(1, 'Nova Distributors', '9876501234', 'nova@store.com'),
(2, 'Brightline Supplies', '9867401256', 'brightline@store.com'),
(3, 'Metro Wholesale', '9856302147', 'metro@store.com');
```

10. Insert Product Records

```
INSERT INTO Product VALUES
(301, 'Wireless Keyboard', 'Accessories', 1250.00, 1),
(302, 'USB Microphone', 'Audio', 2450.00, 2),
(303, 'LED Monitor', 'Display', 9800.00, 3),
(304, 'Laptop Stand', 'Accessories', 1650.00, 1);
```

11. Insert Inventory Records

```
INSERT INTO Inventory VALUES
(21, 301, 18, 5, 'Available'),
(22, 302, 7, 4, 'Available'),
(23, 303, 3, 6, 'Low Stock'),
(24, 304, 0, 3, 'Out of Stock');
```

The sample records represent a different inventory scenario while keeping the same database-management approach.

12. Display Seller Records

```
SELECT * FROM Seller;
```

13. Display Product Records

```
SELECT * FROM Product;
```

14. Display Inventory Records

```
SELECT * FROM Inventory;
```

15. Display Products with Seller Names

```
SELECT
    Seller.seller_name,
    Product.product_name,
    Product.category,
    Product.price
FROM Seller
INNER JOIN Product
ON Seller.seller_id = Product.seller_id;
```

16. Display Products with Available Stock

```
SELECT
    Product.product_name,
    Inventory.stock_quantity
FROM Product
INNER JOIN Inventory
ON Product.product_id = Inventory.product_id
WHERE Inventory.stock_quantity > 0
AND Inventory.status = 'Available';
```

17. Display Low-Stock Products

```
SELECT
    Product.product_name,
    Inventory.stock_quantity,
    Inventory.reorder_level
FROM Product
INNER JOIN Inventory
ON Product.product_id = Inventory.product_id
WHERE Inventory.status = 'Low Stock';
```

18. Generate Complete Inventory Report

```
SELECT
    Product.product_id,
    Product.product_name,
    Product.category,
    Inventory.stock_quantity,
    Inventory.reorder_level,
    Inventory.status
FROM Product
INNER JOIN Inventory
ON Product.product_id = Inventory.product_id;
```

19. Update Inventory Quantity

```
UPDATE Inventory
SET stock_quantity = 12,
    status = 'Available'
WHERE product_id = 303;
```

20. Remove a Product

The dependent inventory row should be deleted first because the Product table is referenced by Inventory.

```
DELETE FROM Inventory
WHERE product_id = 304;

DELETE FROM Product
WHERE product_id = 304;
```

21. Sample Output

Product	Category	Quantity	Status
Wireless Keyboard	Accessories	18	Available
USB Microphone	Audio	7	Available
LED Monitor	Display	3	Low Stock
Laptop Stand	Accessories	0	Out of Stock

22. Conclusion

The Seller and Inventory Management System provides a structured way to maintain seller, product, and stock records. Primary and foreign keys connect the tables and preserve relationships between sellers, products, and inventory. SQL commands can be used to insert new records, modify stock levels, remove outdated products, join related information, and identify products that are available, low in stock, or unavailable.

The system therefore supports organized inventory monitoring and makes it easier to prepare useful stock reports for day-to-day store operations.